TRUSTMEM: Learning Trustworthy Memory Consolidation for LLM Agents with Long-Term Memory

Tianyu Yang* **Sudipta Paul** **Vijay Srinivasan** **Vivek Kulkarni** **Srinivas Chappidi**
AI Center-Mountain View, Samsung Electronics
tyang4@nd.edu sudipta.paul@samsung.com

Abstract

Large language model (LLM) agents rely on long-term memory to support extended interactions and personalized assistance beyond finite context windows. Existing memory agents actively update external memory through *generated write, revise*, and *delete* operations, but these updates may omit important information, corrupt existing memory, or introduce unsupported hallucinated content. Once stored, such errors become persistent system-state failures that can affect future reasoning and generation. In this paper, we propose **TRUSTMEM**, a framework designed to improve the trustworthiness of memory consolidation. TrustMem relies on a Memory Transition Verifier to evaluate the transition process of memory updates in terms of coverage, preservation, and faithfulness. It further constructs preference pairs among candidate updates under the same memory state, enabling preference-guided reinforcement learning to directly optimize memory updating behaviors. Extensive experiments demonstrate that TRUSTMEM improves both memory utility and reliability: it achieves state-of-the-art results across MemoryAgentBench, HaluMem, and the Mem-α validation set, improves HaluMem memory extraction by 12.14 F1 points, and reduces transition-level omission, corruption, and hallucination by 40.1%, 79.1%, and 50.0%, respectively, compared with the strongest baseline for each error type.

1 Introduction

As Large Language Models (LLMs) evolve into intelligent agents capable of long-term interaction and personalized service, long-term memory has become a pivotal component (Shinn et al., 2023; Park et al., 2023). By maintaining an external memory store, agents can transcend finite context windows to preserve user preferences, historical facts, and reasoning trajectories (Zhong et al., 2024; Lu et al., 2023; Packer et al., 2023). Recent advancements (Yu et al., 2025; Wang et al., 2025b; Yan et al., 2025; Yue et al., 2026; Zhou et al., 2025) have moved beyond passive retrieval toward active memory management, where agents actively maintain and revise memory over time. Specifically, a memory agent typically processes streaming inputs in chunks: at each step, it retrieves relevant past contexts and executes structured memory-editing actions, such as inserting new insights, updating existing records, or deleting obsolete information, to consolidate its persistent state. While this flexibility enhances controllability, it raises a critical concern: *can we trust the autonomously generated memory updates?*

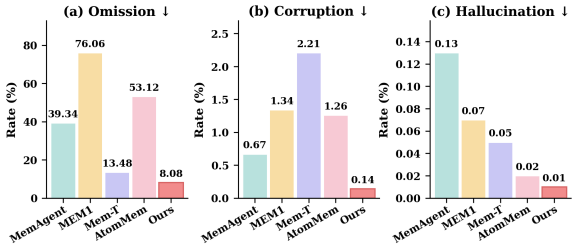


Figure 1: **Transition-level error rates of existing memory agents and TRUSTMEM.** TRUSTMEM achieves the lowest omission, corruption, and hallucination rates, reducing errors by 40.1%, 79.1%, and 50.0% over the strongest baseline for each error type.

As shown in Figure 2, representative memory agents can introduce persistent errors during memory consolidation. These examples are drawn from outputs of Mem1 (Zhou et al., 2025) and Mem-T (Yue et al., 2026) under the same memory-consolidation setting used in our evaluation. For instance, the first row shows a **Corruption** error, where an existing method incorrectly places Voltaire in the 17th century despite the input associating him with the 18th century. The second row illustrates **Omission**, where key split and reunifica-

*Currently at University of Notre Dame. Work done during an internship at AI Center-Mountain View, Samsung Electronics.

Findings of the Association for Computational Linguistics: EMNLP 2026, pages 51948–51965
October 24-29, 2026 ©2026 Association for Computational Linguistics

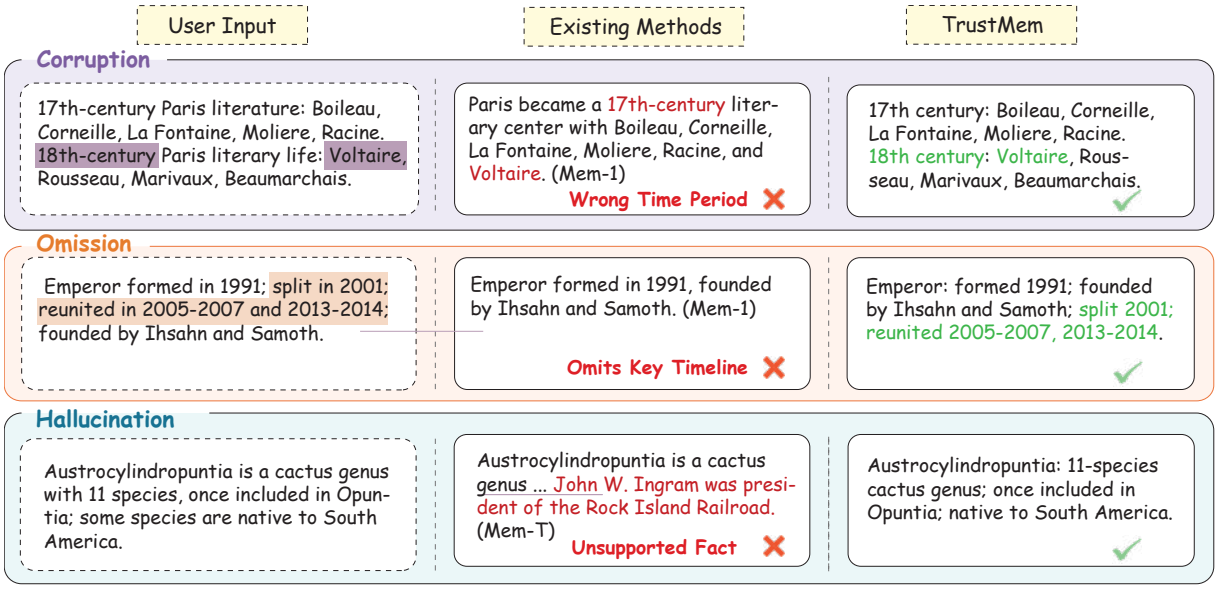


Figure 2: **Comparison of memory consolidation results produced by existing methods and TRUSTMEM.** Compared with Mem1 (Zhou et al., 2025) and Mem-T (Yue et al., 2026), TRUSTMEM better preserves input evidence and avoids corruption, omission, and hallucination when converting user inputs into persistent memory.

tion timelines are dropped from memory. The third row shows **Hallucination**, where an unsupported statement about John W. Ingram is introduced despite being grounded in neither the input nor the previous memory state. Since these errors can compound over time, the reliability of a memory system cannot be judged solely by terminal performance. Once a flawed entry is consolidated, it may be repeatedly retrieved and amplified in subsequent interactions, persistently contaminating the agent's reasoning chain (Chen et al., 2025). Consequently, the core challenge of long-term memory systems has shifted from merely "retaining" information to *building trustworthy memory*.

Furthermore, existing supervisory signals struggle to address these issues due to a severe credit assignment gap (Luo et al., 2025). Most training paradigms (Wang et al., 2025b; Yu et al., 2025; Yue et al., 2026; Yan et al., 2025) rely on terminal scalar rewards—such as final task success or tool-calling validity—which act on the trajectory level rather than the transition level for memory updates. These global signals are too coarse to pinpoint exactly when or where a faulty memory update occurred within a long sequence of updates. An incorrect final answer could stem from an early omission, a destructive overwrite in mid-stream, or a simple reasoning flaw at the final step. Conversely, a correct answer may mask unsafe intermediate updates that did not trigger a failure in the current turn but remain latent risks for future tasks. Therefore, relying solely on terminal outcomes fails to provide the granular supervision necessary to explicitly correct the agent's specific memory-editing behaviors.

To address this challenge, we shift the focus from evaluating only the final memory outcome to supervising how memory is constructed step by step. The agent processes the input chunk by chunk and executes structured memory-editing actions, such as writing new information, revising existing entries, or pruning outdated content. Each action sequence induces a local transition from the previous memory state to the updated one, enabling us to inspect whether each intermediate update is trustworthy before it becomes persistent.

Based on this formulation, we propose **TRUSTMEM**, a framework for training memory agents to perform trustworthy memory consolidation at the transition level. In particular, TRUSTMEM introduces a **Memory Transition Verifier** to evaluate each memory update along coverage, preservation, and faithfulness, and further proposes **Transition-Ranked GRPO**, built upon GRPO (Shao et al., 2024), to optimize the memory-update policy. By ranking candidate updates under the same memory state, TRUSTMEM provides fine-grained supervision for memory-editing behaviors while maintaining the downstream utility of the final memory.

We evaluate TRUSTMEM across three settings: MemoryAgentBench (Hu et al., 2025),

HaluMem (Chen et al., 2025), and the Mem-α (Wang et al., 2025b) validation set, covering accurate retrieval, test-time learning, long-range understanding, and memory reliability. Extensive experiments demonstrate that TRUSTMEM achieves state-of-the-art results across these settings, improving over the strongest baseline by 6.5 points on MemoryAgentBench and 12.14 F1 points in HaluMem memory extraction. Beyond final task performance, Figure 1 shows that TRUSTMEM achieves the lowest transition-level error rates across omission, corruption, and hallucination. Compared with the strongest baseline for each error type, TRUSTMEM reduces omission, corruption, and hallucination by 40.1%, 79.1%, and 50.0%, respectively. These results demonstrate that TRUSTMEM improves not only memory utility, but also operation-level reliability and transition-level safety.

2 Related Works

Long-Term Memory Systems for LLM Agents. Long-term memory enables LLM agents to preserve information beyond finite context windows. Early systems store interactions, reflections, or user-specific notes in external memory, including Generative Agents (Park et al., 2023), Reflexion (Shinn et al., 2023), MemoryBank (Zhong et al., 2024), and MemoChat (Lu et al., 2023). System-level frameworks such as MemGPT (Packer et al., 2023), MemoryOS (Kang et al., 2025), Mem0 (Chhikara et al., 2025), A-MEM (Xu et al., 2025), MIRIX (Wang and Chen, 2025), Zep (Rasmussen et al., 2025), and LightMem (Fang et al., 2025) improve memory management through external storage, extraction, organization, graph structure, retrieval, or efficiency. Another line explores parametric or latent memory, including MemoryLLM (Wang et al., 2024), M+ (Wang et al., 2025a), Titans (Behrouz et al., 2024), and MemGen (Zhang et al., 2025). These works primarily study how memory is stored, retrieved, compressed, or represented, whereas TRUSTMEM focuses on whether each generated memory update is trustworthy before it gets stored.

RL-based Memory Agents. Recent methods formulate memory management as a learnable decision process. MemAgent (Yu et al., 2025) trains LLMs to process long inputs segment by segment and maintain compact memory; MEM1 (Zhou et al., 2025) studies the interaction between memory construction and reasoning; Memory-R1 (Yan et al., 2025) learns explicit ADD, UPDATE, DELETE, and NOOP operations; and Mem-α (Wang et al., 2025b) optimizes sequential memory construction over multi-component memories with RL. AgeMem (Yu et al., 2026), ReMemR1 (Shi et al., 2025), and Mem-T (Yue et al., 2026) further explore unified memory actions, revisitable memory, and denser credit assignment. While these methods demonstrate that RL can improve memory utility, they mainly optimize final task performance or trajectory-level rewards. TRUSTMEM instead introduces transition-level verification and preference ranking to supervise the safety of each local memory update.

Reliable Memory Agents. As memory becomes persistent and editable, reliability failures can accumulate across interactions. Benchmarks such as LoCoMo (Maharana et al., 2024), LongMemEval (Wu et al., 2024), and MemoryAgentBench (Hu et al., 2025) evaluate long-term memory through multi-session QA, temporal reasoning, knowledge updates, test-time learning, long-range understanding, or selective forgetting. HaluMem (Chen et al., 2025) further analyzes operation-level hallucination during memory extraction, updating, and memory-based QA. Related studies examine memory injection attacks (Dong et al., 2025; Tian et al., 2026), memory admission control (Zhang et al., 2026), provenance-aware memory (Zhu et al., 2026), and governance of evolving memory (Lam et al., 2026). Unlike these evaluation- or defense-oriented works, TRUSTMEM directly optimizes the memory-update policy with transition-level verification, reducing omission, corruption, and hallucination before unsafe updates become persistent.

3 Method

3.1 Overview of TRUSTMEM Framework

We formulate memory consolidation as a continual memory-update problem. At each step, the memory agent receives a new input chunk from a long-form interaction stream, such as a document segment, a dialogue turn, or a task demonstration segment. We denote the input stream as $C = \{c_1, c_2, \ldots, c_T\}$, where c_t is the newly observed chunk at step t. The chunks are processed in chronological order and consolidated into an evolving external memory.

Let M_{t-1} denote the persistent memory state before processing chunk c_t. This memory state stores useful information accumulated from pre-

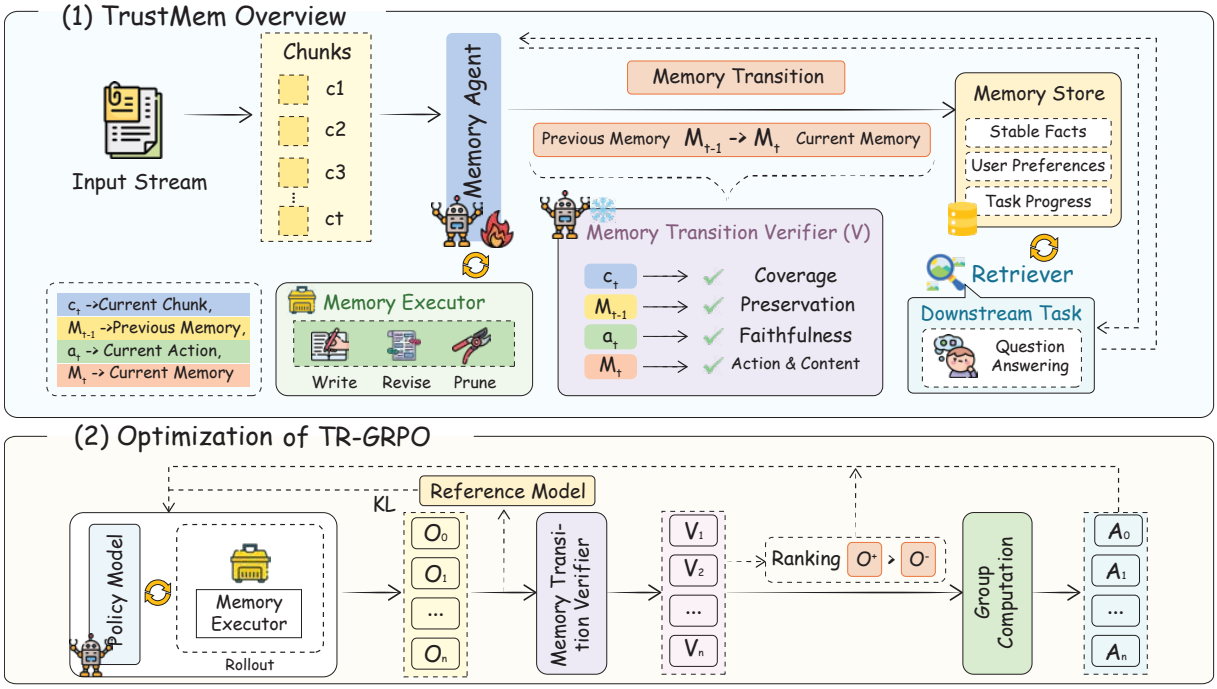


Figure 3: **Overview of TRUSTMEM. Top:** TRUSTMEM performs continual memory consolidation over an input stream. At step t, the memory agent observes the current chunk c_t and the accumulated memory state M_{t-1}, generates memory-editing actions, and updates the memory state to M_t through the Memory Executor. The resulting transition is evaluated by the Memory Transition Verifier for coverage, preservation, and faithfulness before the updated memory is used for retrieval and downstream tasks. **Bottom:** TRUSTMEM optimizes the memory agent with Transition-Ranked GRPO, where verifier scores from multiple rollouts are used for both scalar reward computation and transition-level preference ranking.

vious chunks $c_1, \ldots, c_{t-1}$, rather than the content of only one chunk. At step t, the agent observes (c_t, M_{t-1}) and generates a sequence of structured memory-editing actions $a_t \sim \pi_\theta(\cdot \mid c_t, M_{t-1})$. Thus, the agent handles one chunk at each step, while the memory state represents global accumulated memory after all processed chunks so far.

To make memory updates executable and auditable, we define a unified action space $\mathcal{A} = \{\text{WRITE}, \text{REVISE}, \text{PRUNE}\}$. Specifically, WRITE inserts new memory items when the current chunk contains useful information not yet stored in memory; REVISE updates or merges existing memory entries when the current chunk provides corrections, refinements, or complementary evidence; and PRUNE removes memory entries that become invalid, redundant, or no longer useful for downstream tasks based on the current evidence and the existing memory state. The agent decides whether an update is needed by generating one or more of these actions conditioned on (c_t, M_{t-1}). If the current chunk does not introduce new useful information, correct previous memory, or make any existing entry invalid or redundant, the agent may output no action.

A deterministic memory executor $\mathcal{E}$ then applies a_t to the memory store, yielding the updated state $M_t = \mathcal{E}(M_{t-1}, a_t)$. This incremental construction defines a local memory transition $z_t = (c_t, M_{t-1}, a_t, M_t)$.

Since each update alters the agent's persistent memory state, we introduce a Memory Transition Verifier V to evaluate the trustworthiness of each local transition, yielding a score $v_t = V(z_t) \in [0, 1]$. As detailed in Section 3.2, the verifier checks if the update preserves important information from the current chunk, maintains valid prior memory, and avoids unsupported hallucinated content.

After all T chunks are processed, the terminal memory M_T is used for downstream tasks. For a query q, a fixed retriever fetches relevant records $S_q = \mathcal{R}(M_T, q)$, and an answer model generates a prediction $\hat{y} = g_{\text{ans}}(q, S_q)$, which is compared against the reference answer y to compute the task reward. Our overall objective is to optimize the policy π_θ so that the agent constructs compact and useful terminal memory while ensuring that each intermediate memory transition is trustworthy.

3.2 Memory Transition Verifier

Because memory construction is cumulative, relying solely on terminal downstream accuracy suffers from a severe credit assignment gap. An incorrect final answer may stem from an early omission, a later destructive edit, or a failure of the answer model. Conversely, a correct final answer does not guarantee that all intermediate memory updates are safe, since silent errors may remain in the persistent memory state and affect future tasks.

To provide fine-grained supervision, we introduce a **Memory Transition Verifier** V that evaluates each local memory transition defined in Section 3.1. Rather than judging the final answer, the verifier acts as a state-transition evaluator and checks whether the update from M_{t-1} to M_t is trustworthy after processing the current chunk.

For each transition $z_t = (c_t, M_{t-1}, a_t, M_t)$, the verifier is instantiated as a frozen LLM and receives a compact view of the transition, including the current chunk, the generated memory-editing actions, and the memory entries touched or retrieved by these actions. This avoids feeding the full memory bank while still providing the evidence needed to judge the local update. The simplified verifier prompt is shown in Figure 5 in the Appendix.

Specifically, the verifier evaluates each transition along three diagnostic dimensions: **coverage**, which checks whether important information from the current chunk is preserved; **preservation**, which checks whether valid prior memory is maintained without unjustified deletion or distortion; and **faithfulness**, which checks whether newly added or modified memory content is supported by the current chunk or the previous memory state. These judgments are aggregated into a scalar trustworthiness score $v_t = V(z_t) \in [0, 1]$, which is used as the transition-level verification reward.

By directly evaluating intermediate memory transitions, the verifier provides local supervision for memory-editing behavior and helps prevent omission, corruption, and hallucination before they become persistent memory errors.

3.3 Multi-granularity Reward Design

Training reliable memory agents requires supervision at different granularities. Sample-level rewards evaluate the utility and compactness of the final memory after the full input stream has been processed, while transition-level rewards evaluate whether each local memory update is executable, meaningful, and trustworthy. We therefore design a multi-granularity reward that combines sample-level rewards with transition-level rewards.

Sample-Level Rewards. These rewards evaluate the terminal memory state $M_T^{(i)}$ after all chunks in sample i are processed.

Task Reward (R_{task}) measures downstream utility: given questions $Q^{(i)}$, a fixed retriever $\mathcal{R}$ and answer model g_{ans} generate predictions $\hat{y}$, and the reward is the average metric, such as F1 or LLM-as-a-judge score, against reference answers y.

Efficiency Reward (R_{eff}) penalizes memory bloat to prevent excessive copying, computed as $1 - (|M_T^{(i)}| / \sum_t |c_t^{(i)}|)$, encouraging compact memory.

Transition-Level Rewards. These rewards evaluate each local memory transition $M_{t-1} \to M_t$.

Action Execution (R_{act}) checks whether the generated structured actions are parseable and executable by the memory executor $\mathcal{E}$.

Content Type (R_{type}) uses a lightweight analyzer J_{type} to ensure that the modified content is semantically specific and non-empty.

Verifier Reward (R_{ver}) uses the Memory Transition Verifier V from Section 3.2 to evaluate the semantic safety of the transition, defined as $R_{\text{ver},t}^{(i)} = V(z_{i,t})$.

Aggregating the above, we define the total reward for the t-th memory update of sample i as

$$\begin{aligned} R_{i,t} = \; & R_{\text{task}}^{(i)} + \lambda_{\text{eff}} R_{\text{eff}}^{(i)} \\ & + \lambda_{\text{act}} R_{\text{act},t}^{(i)} + \lambda_{\text{type}} R_{\text{type},t}^{(i)} + \lambda_{\text{ver}} R_{\text{ver},t}^{(i)}. \end{aligned} \tag{1}$$

This reward combines final memory utility, memory compactness, action executability, content specificity, and transition-level trustworthiness. The sample-level terms provide global supervision over the final constructed memory, while the transition-level terms provide local supervision for each memory update.

3.4 Transition-Ranked GRPO

Building on the multi-granularity reward, we propose **Transition-Ranked GRPO**, which combines scalar reward optimization with verifier-guided transition ranking. While the verifier score can be used as a transition-level reward in Eq. (1), its absolute magnitude may not always be well calibrated. However, in our setting, verifier scores are used more robustly as relative preferences among candidate updates generated under the same state than as fully calibrated absolute values. Therefore, we

Method	AR			TTL			LRU	Avg.
	SQuAD	HotpotQA	PerLTQA	TREC-C	NLU	PubMed	BookSum	
Long-Context	74.2	**85.2**	60.5	62.3	70.8	53.3	5.2	58.8
RAG-Top2	76.2	84.9	62.3	61.2	50.8	57.0	4.2	56.7
MemAgent (Yu et al., 2025)	52.6	66.0	41.5	53.5	57.4	61.6	8.6	48.7
MEM1 (Zhou et al., 2025)	12.0	16.4	16.0	47.5	33.2	22.6	6.6	22.0
Mem-α (Wang et al., 2025b)	78.6	83.2	**65.9**	66.6	65.8	54.5	18.7	61.9
TRUSTMEM	**80.6**	84.9	63.6	**72.4**	**73.8**	**64.0**	**22.6**	**66.3**

Table 1: Experiment results on the Mem-α validation set. The top-2 results are highlighted.

Method	AR			TTL					LRU	Avg.
	Single-Doc	Multi-Doc	LME(S)	TREC-C	NLU	TREC-F	Clinic	Banking77	InfBench	
Long-Context	28.0	27.0	29.2	64.0	74.0	34.0	86.0	77.0	12.5	46.1
RAG-Top2	69.0	45.0	**58.1**	69.0	65.0	21.0	70.0	75.0	6.5	50.2
MemAgent (Yu et al., 2025)	51.0	52.0	23.0	62.0	78.0	30.0	**88.0**	83.0	10.9	53.1
MEM1 (Zhou et al., 2025)	23.0	33.0	13.0	57.0	**80.0**	28.0	48.0	76.0	7.0	40.6
Mem-T (Yue et al., 2026)	56.0	60.0	43.3	75.0	48.0	27.0	43.0	79.0	14.1	49.5
AtomMem (Huo et al., 2026)	44.0	45.0	35.3	51.0	67.0	11.0	46.0	66.0	**19.3**	19.3
Mem-α (Wang et al., 2025b)	74.0	**68.0**	52.0	71.0	71.0	**41.0**	73.0	70.0	12.9	59.2
TRUSTMEM	**83.0**	64.0	55.0	**85.0**	79.2	**41.0**	84.0	**84.0**	15.9	**65.7**

Table 2: Experiment results on MemoryAgentBench. The top-2 results are highlighted.

further use verifier scores to construct preference pairs among candidate transitions and optimize the policy with an additional ranking objective.

Global Trajectory Optimization via GRPO. We adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024) to optimize the memory policy. Specifically, for each training sample i, we sample a group of G rollouts from the old policy $\pi_{\theta_{\text{old}}}$. At each chunk step t within rollout g, we compute the scalar reward $R_{i,t}^{(g)}$ from Eq. (1). The group-normalized advantage is computed as $\hat{A}_{i,t}^{(g)} = (R_{i,t}^{(g)} - \mu_{i,t})/(\sigma_{i,t} + \epsilon_A)$, where $\mu_{i,t}$ and $\sigma_{i,t}$ are the mean and standard deviation of rewards across the G rollouts for the same chunk step. Letting $r_{i,t,l}^{(g)}(\theta)$ denote the importance ratio for the l-th generated token, the GRPO objective over the memory construction trajectory is formulated as:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{i,t,g,l}\Big[\min\big(r_{i,t,l}^{(g)}(\theta)\hat{A}_{i,t}^{(g)}, \text{clip}\big(r_{i,t,l}^{(g)}(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_{i,t}^{(g)}\big)\Big]. \quad (2)$$

Verifier-Guided Pair Construction. For each training sample i, the G rollouts produce multiple candidate memory transitions at each chunk step t, denoted as $\mathcal{Z}_{i,t} = \{z_{i,t}^{(1)}, \ldots, z_{i,t}^{(G)}\}$. These candidate transitions are generated from the same current chunk and previous memory state, but may contain different memory-editing actions. Because the verifier V evaluates each transition independently, it induces a ranking over $\mathcal{Z}_{i,t}$. We select the transitions with the highest and lowest verifier scores as the preferred transition z^+ and rejected transition z^-, respectively, provided their score gap exceeds a margin δ. These preference pairs capture local differences in transition quality and provide additional supervision beyond the scalar reward.

Local Preference Ranking via Verifier. Given the preference pairs (z^+, z^-), we optimize a DPO-style ranking loss to inject step-level semantic supervision. Because z^+ and z^- are generated for the same chunk and previous memory state, they share the same prompt $p_{i,t}$ and differ only in the generated memory-editing actions. Let o^+ and o^- denote the preferred and rejected action sequences.

We define the length-normalized log-probability margin as $\Delta_\theta = \ell_\theta(o^+ \mid p_{i,t}) - \ell_\theta(o^- \mid p_{i,t})$, where $\ell_\theta(o \mid p)$ denotes the length-normalized log probability of a generated response o given prompt p. The ranking loss is:

$$\mathcal{L}_{\text{rank}} = -\log\sigma\left(\beta_{\text{rank}}\left[\Delta_\theta - \Delta_{\text{ref}}\right]\right), \quad (3)$$

where $\Delta_{\text{ref}} = \ell_{\text{ref}}(o^+ \mid p_{i,t}) - \ell_{\text{ref}}(o^- \mid p_{i,t})$ is the corresponding margin under the reference policy.

Joint Objective. We use a frozen reference policy π_{ref} to regularize policy updates with a KL penalty. This KL term is applied as an actor-side auxiliary loss, rather than being added to the scalar

Method	Memory Extraction				Memory Updating			Question Answering		
	R↑	Weighted R↑	Acc.↑	F1↑	C↑	H↓	O↓	C↑	H↓	O↓
Mem0 (Chhikara et al., 2025)	42.91	65.03	60.86	57.31	25.50	0.45	74.02	53.02	19.17	27.81
Memobase (Ye et al., 2025)	14.55	25.88	32.29	25.13	5.20	0.55	94.25	35.33	29.97	34.71
Supermemory (Shah et al., 2025)	41.53	64.76	60.83	56.90	16.37	1.15	82.47	54.07	22.24	23.69
Zep (Rasmussen et al., 2025)	–	–	–	–	**47.28**	**0.42**	52.31	55.47	21.92	22.62
MemAgent (Yu et al., 2025)	12.39	15.31	82.02	22.03	11.66	4.55	83.78	42.11	28.70	28.12
Mem-T (Yue et al., 2026)	36.60	46.85	67.13	52.69	45.53	3.35	51.12	**59.43**	26.11	13.98
AtomMem (Huo et al., 2026)	0.28	0.39	**87.50**	0.56	2.14	4.29	93.57	31.27	**5.56**	64.00
TRUSTMEM	**56.73**	**67.45**	67.81	**69.45**	46.84	0.53	**48.64**	55.22	6.07	**12.46**

Table 3: Experiment results on HaluMem Benchmark. We report operation-level metrics for memory extraction, memory updating, and memory-based question answering. R, Weighted R, Acc., and F1 evaluate memory extraction quality; C, H, and O denote correctness, hallucination, and omission, respectively.

Methods	AR	TTL	LRU	Overall
Ablation of Reward Components				
w/o R_{act}	67.4	52.6	15.9	54.0
w/o R_{eff}	71.1	66.9	18.8	61.8
Ablation of Transition Verification				
w/o R_{ver}	72.1	58.3	20.6	58.7
Ablation of Optimization Strategy				
PRM-style GRPO	75.8	57.6	21.5	60.3
TrustMem (TR-GRPO)	76.4	70.1	22.6	66.3

Table 4: Ablation studies on reward components, transition verification, and optimization strategy.

reward used for advantage estimation. Here, $\pi_{\theta_{\text{old}}}$ denotes the rollout policy used in the GRPO importance ratio, while π_{ref} denotes the frozen reference policy used for KL regularization and the ranking margin.

The final training objective combines scalar reward optimization, KL regularization, and transition-level ranking:

$$\mathcal{L} = -\mathcal{J}_{\text{GRPO}} + \beta_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{rank}}\mathcal{L}_{\text{rank}}. \quad (4)$$

Here, $\mathcal{L}_{\text{KL}}$ denotes the token-level KL regularization loss, while β_{KL} and λ_{rank} control the weights of KL regularization and transition-level ranking, respectively.

4 Experiment

We provide experimental settings, including dataset statistics, baseline descriptions, implementation details, and evaluation protocols, in Appendix A.

4.1 Experiments on Memory Utility

Table 1 reports results on the Mem-α validation set. TrustMem achieves the best average score of 66.3, outperforming the strongest baseline Mem-α by 4.4 points. The improvement is especially clear on Test-Time Learning, where TrustMem obtains the best scores on TREC-C (72.4), NLU (73.8), and PubMed (64.0). TrustMem also improves BookSum from 18.7 to 22.6, showing better long-range memory utility. These gains indicate that transition-level verification helps the agent preserve useful task-specific and long-range information during memory construction.

Table 2 evaluates out-of-distribution generalization on MemoryAgentBench. TrustMem again achieves the best average score of 65.7, improving over Mem-α by 6.5 points and outperforming Long-Context and RAG-Top2 by 19.6 and 15.5 points, respectively. TrustMem performs strongly on both retrieval and test-time learning tasks, achieving the best score on Single-Doc (83.0), TREC-C (85.0), and Banking77 (84.0). These results show that TrustMem learns a more robust memory update policy than simply retaining long context or retrieving top-ranked chunks. Overall, these results demonstrate that reliable local memory transitions improve both validation performance and out-of-distribution memory utility, with detailed analysis provided in Appendix B.

4.2 Experiments on HaluMem

Table 3 evaluates operation-level memory reliability on HaluMem-Medium, covering memory extraction, memory updating, and memory-based question answering. TrustMem achieves the best memory extraction recall (56.73), weighted recall (67.45), and F1 (69.45), improving the strongest prior F1 score by 12.14 points. Although AtomMem obtains the highest extraction accuracy, its very low recall and F1 suggest an overly conservative extraction behavior that misses most useful memories. In contrast, TrustMem achieves a stronger coverage-correctness trade-off, resulting in the best overall extraction quality. For memory updating and memory-based question answering,

TrustMem obtains the lowest omission rates while maintaining competitive correctness and low hallucination. These results show that transition-level verification and preference-guided optimization improve intermediate memory reliability beyond final task performance. A detailed stage-wise analysis is provided in Appendix C.

4.3 Ablation Study

As shown in Table 4, we perform ablation studies on the Mem-α validation set to quantitatively evaluate the impact of Action Execution Reward (R_{act}), Efficiency Reward (R_{eff}), Transition Verifier Reward (R_{ver}), and the Transition-Ranked GRPO optimization strategy.

Effectiveness of Reward Components. Removing R_{act} causes the largest drop, reducing the overall score from 66.3 to 54.0. This shows that executable memory-editing actions are crucial for reliable memory construction. Removing R_{eff} also decreases the overall score to 61.8, indicating that compact memory states help reduce redundancy and improve downstream memory utility.

Effectiveness of the Memory Transition Verifier. Removing R_{ver} reduces the overall score from 66.3 to 58.7, with a clear drop on TTL from 70.1 to 58.3. This confirms that the Memory Transition Verifier provides important transition-level supervision. By checking local memory updates for omission, corruption, and hallucination, the verifier helps prevent unsafe updates from entering the persistent memory state.

Effectiveness of Transition-Ranked GRPO. Compared with PRM-style GRPO, TrustMem with Transition-Ranked GRPO improves the overall score from 60.3 to 66.3. The improvement is especially large on TTL, increasing from 57.6 to 70.1. This suggests that ranking candidate memory transitions provides a stronger learning signal than directly using verifier scores as scalar rewards, leading to safer and more useful memory updates.

4.4 Reliability of Memory Consolidation

We evaluate transition-level reliability by measuring three semantic failure modes: omission, corruption, and hallucination. Using generated trajectories from the 463-instance Mem-α validation set, we construct chunk-level transition records for each method and use GPT-4o-mini (Hurst et al., 2024) with temperature 0 as an LLM judge. The judge follows the prompt and definitions in Appendix D; we report the percentage of transitions containing each major error type.

As shown in Figure 1, omission is the dominant error type because memory construction is inherently compressive, while corruption and hallucination require actively writing incorrect or unsupported facts. Thus, comparisons should be made within each error type across methods. TrustMem achieves the lowest error rates across all categories, with 8.08% omission, 0.14% corruption, and 0.01% hallucination, demonstrating that transition-level verification reduces missing, distorted, and unsupported memory content.



Figure 4: Training reward dynamics of TrustMem and the variant without Transition-Ranked GRPO.

4.5 Training Reward Dynamics

Figure 4 compares TrustMem with a variant trained without Transition-Ranked GRPO. Both methods improve in the early stage, suggesting that task-level rewards can learn basic memory-editing behaviors. However, the variant without transition-ranked optimization later becomes unstable and shows a sharp drop in both task and total rewards, while TrustMem maintains higher rewards and converges to a stable plateau. This indicates that coarse scalar rewards alone are insufficient for reliable memory consolidation. By ranking candidate memory transitions with verifier scores, Transition-Ranked GRPO provides finer-grained supervision over memory state changes, encouraging trustworthy updates and stabilizing training.

4.6 Case Study

We present representative case studies in Figure 2, with additional examples provided in Table 5 in the Appendix. These examples cover transition-level failures: corruption, omission, and hallucination.

Existing memory agents often produce unreliable memory updates. Mem1 reverses input semantics by storing a negation-reversed fact; AtomMem omits all key status-symbol facts; and MemAgent hallucinates user events from labeled command examples. Such errors are written into the memory

and may affect future retrieval and reasoning.

In contrast, TRUSTMEM preserves key evidence and avoids unsupported memory edits, demonstrating more faithful transition-level memory consolidation.

5 Conclusion

In this paper, we introduce TRUSTMEM, a framework for learning trustworthy memory consolidation in LLM agents with long-term memory. Instead of relying only on final task outcomes, TRUSTMEM supervises intermediate memory transitions through a Memory Transition Verifier and optimizes memory-editing behavior with Transition-Ranked GRPO. Experiments on MemoryAgentBench, HaluMem, and the Mem-α show that TRUSTMEM improves both memory utility and operation-level reliability. The results further demonstrate that transition-level verification helps reduce omission, corruption, and hallucination, highlighting its importance for reliable long-term memory systems.

Limitations

A key limitation of TRUSTMEM is that our current experiments mainly focus on text-based long-term memory benchmarks. However, in real-world agent applications, user inputs and memory evidence are often multimodal, involving not only textual interactions but also images, videos, documents, and other contextual signals. We plan to extend our framework to multimodal memory agents and vision-language models, enabling more faithful and trustworthy consolidation of multimodal memory for open-ended personalized applications.

References

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. 2024. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd workshop on natural language processing for conversational AI*, pages 38–45.

Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchi Li, Feiyu Xiong, and Zhiyu Li. 2025. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506*.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

Franck Dernoncourt and Ji-Young Lee. 2017. Pubmed 200k rct: a dataset for sequential sentence classification in medical abstracts. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 308–313.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory injection attacks on llm agents via query-only interaction. *arXiv preprint arXiv:2503.03704*.

Yiming Du, Hongru Wang, Zhengyi Zhao, Bin Liang, Baojun Wang, Wanjun Zhong, Zezhong Wang, and Kam-Fai Wong. 2024. Perltqa: A personal long-term memory dataset for memory classification, retrieval, and fusion in question answering. In *Proceedings of the 10th SIGHAN Workshop on Chinese Language Processing (SIGHAN-10)*, pages 152–164.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, and 1 others. 2025. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*.

Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*.

Yupeng Huo, Yaxi Lu, Zhong Zhang, Haotian Chen, and Yankai Lin. 2026. Atommem: Learnable dynamic agentic memory with atomic memory operation. *arXiv preprint arXiv:2601.08323*.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25972–25981.

Wojciech Kryściński, Nazneen Rajani, Divyansh Agarwal, Caiming Xiong, and Dragomir Radev. 2022. Booksum: A collection of datasets for long-form narrative summarization. In *Findings of the association for computational linguistics: EMNLP 2022*, pages 6536–6558.

Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. 2026. Governing evolving memory in llm agents: Risks, mechanisms, and the stability and safety governed memory (ssgm) framework. *arXiv preprint arXiv:2603.11768*.

Stefan Larson, Anish Mahendran, Joseph J Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K Kummerfeld, Kevin Leach, Michael A Laurenzano, Lingjia Tang, and 1 others. 2019. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1311–1316.

Xin Li and Dan Roth. 2006. Learning question classifiers: the role of semantic information. *Natural Language Engineering*, 12(3):229–249.

Junru Lu, Siyu An, Mingbao Lin, Gabriele Pergola, Yulan He, Di Yin, Xing Sun, and Yunsheng Wu. 2023. Memochat: Tuning llms to use memos for consistent long-range open-domain conversation. *arXiv preprint arXiv:2308.08239*.

Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K Qiu, and Yuqing Yang. 2025. Agent lightning: Train any ai agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870.

Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 2383–2392.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.

Dhravya Shah, Mahesh Sanikommu, Yash, and 1 others. 2025. Supermemory. https://supermemory.ai/. Accessed: 2025-11-05.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Yaorui Shi, Yuxin Chen, Siyuan Wang, Sihang Li, Hengxing Cai, Qi Gu, Xiang Wang, and An Zhang. 2025. Look back to reason forward: Revisitable memory for long-context llm agents. *arXiv preprint arXiv:2509.23040*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.

Hanling Tian, Zeyang Sha, Jingying Wang, Yuhang Liu, Zhehao Huang, and Xiaolin Huang. 2026. Injecmem: Memory injection attack on llm agent memory systems.

Yu Wang and Xi Chen. 2025. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*.

Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, and 1 others. 2024. Memoryllm: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*.

Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. 2025a. M+: Extending memoryllm with scalable long-term memory. *arXiv preprint arXiv:2502.00592*.

Yu Wang, Ryuichi Takanobu, Zhiqi Liang, Yuzhen Mao, Yuanzhe Hu, Julian McAuley, and Xiaojian Wu. 2025b. Mem-{\alpha}: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, and 1 others. 2025. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering.

In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380.

Gustavo Ye, Jinjia, Gener, and 1 others. 2025. Memobase. https://github.com/memodb-io/memobase. Accessed: 2025-11-05.

Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and 1 others. 2025. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*.

Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*.

Yanwei Yue, Boci Peng, Xuanbo Fan, Jiaxin Guo, Qiankun Li, and Yan Zhang. 2026. Mem-t: Densifying rewards for long-horizon memory agents. *arXiv preprint arXiv:2601.23014*.

Guibin Zhang, Muxin Fu, and Shuicheng Yan. 2025. Memgen: Weaving generative latent memory for self-evolving agents. *arXiv preprint arXiv:2509.24704*.

Guilin Zhang, Wei Jiang, Xiejiashan Wang, Aisha Behr, Kai Zhao, Jeffrey Friedman, Xu Chu, and Amine Anoun. 2026. Adaptive memory admission control for llm agents. *arXiv preprint arXiv:2603.04549*.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and 1 others. 2024. Infinitebench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15262–15277.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*.

Qiming Zhu, Shunian Chen, Rui Yu, Zhehao Wu, and Benyou Wang. 2026. From lossy to verified: A provenance-aware tiered memory for agents. *arXiv preprint arXiv:2602.17913*.

Input Chunk	Existing Method Memory	TRUSTMEM Memory
Paris became the capital of French literature in the 17th century with Boileau, Corneille, La Fontaine, Moliere, and Racine. In the 18th century, Parisian literary life was dominated by Voltaire, Rousseau, Marivaux, and Beaumarchais.	MEM1. **Stored memory:** Paris became a 17th-century literary center with Boileau, Corneille, La Fontaine, Moliere, Racine, and Voltaire. **Corruption: wrong time period.**	**Ours.** **Stored memory:** Paris literary timeline: 17C—Boileau, Corneille, La Fontaine, Moliere, Racine; 18C—Voltaire, Rousseau, Marivaux, Beaumarchais.
Constantine Monomachos restored the Church of the Holy Sepulchre between 1042 and 1048. Nothing survived of the mosaics covering the walls and dome; Russian abbot Daniel later left a description.	MEM1. **Stored memory:** Daniel described mosaics as lively and covering the walls and dome. **Corruption: negation reversed.**	**Ours.** **Stored memory:** Holy Sepulchre: restored by Constantine Monomachos in 1042–1048; wall/dome mosaics did not survive; Daniel later described the decoration.
Emperor is a Norwegian black metal band formed in 1991. It split in 2001; reunited in 2005–2007 and 2013–2014.It was founded by Ihsahn and Samoth.	MEM1. **Stored memory:** Emperor is a Norwegian black metal band formed in 1991, founded by Ihsahn and Samoth. **Omission: missing split/reunion timeline.**	**Ours.** **Stored memory:** Emperor: Norwegian black metal band, founded 1991 by Ihsahn/Samoth; split 2001; reunions in 2005–2007 and 2013–2014.
"Because the Night" reached #13 on Billboard, number 5 in the UK, and was ranked number 116on *NME*'s Top 150 Singles of All Time.	MEMAGENT. **Stored memory:** "Because the Night" reached #13 on Billboard. **Omission: missing UK peak and NME ranking.**	**Ours.** **Stored memory:** "Because the Night": Billboard #13; UK number 5; NME all-time rank number 116.
Only senators could wear Tyrian purple in ancient Rome; high-ranking Hawaiian chiefs could wear feather cloaks; before the republic, only the Chinese emperor could wear yellow.	ATOMMEM. **Stored memory:** None. **Omission: all status-symbol facts omitted.**	**Ours.** **Stored memory:** Clothing as status marker: Roman senators—Tyrian purple; Hawaiian chiefs—feather cloaks; Chinese emperor—yellow.
Austrocylindropuntia is a cactus genus with 11 recognized species. It was once included in Opuntia; some species are native to South America.	MEMT. **Stored memory:** Austrocylindropuntia is a cactus genus . . . John W. Ingram was president of the Rock Island Railroad. **Hallucination: unsupported railroad fact.**	**Ours.** **Stored memory:** Austrocylindropuntia: 11-species cactus genus; formerly Opuntia; some South American.
RAAF Base Edinburgh is a Royal Australian Air Force military airbase located 25 km north of Adelaide, South Australia.	MEMT. **Stored memory:** RAAF Base Edinburgh is a RAAF airbase . . . EPMD is a hip-hop group; "True North" is a TV episode; Bill Parcells coached the Giants. **Hallucination: unrelated pop-culture facts.**	**Ours.** **Stored memory:** RAAF Base Edinburgh: RAAF military base near Edinburgh, South Australia, 25 km north of Adelaide.
Input contains labeled command examples: "add new email to contacts"; "how is shell stock doing today"; "confirm that the alarm is set for tomorrow".	MEMAGENT. **Stored memory:** John is the user's friend. At 2024-01-10, the user added a new email to contacts and checked Shell stock. **Hallucination: examples treated as real events.**	**Ours.** **Stored memory:** Command-intent examples: add email: label 1; check Shell stock: label 4; confirm alarm: label 42.
Czech verb aspect pairs differ by prefix or suffix; Czech has indicative, imperative, and conditional moods; Czech orthography is phonemic and uses hacek marks.	ATOMMEM. **Stored memory:** Czech verb aspects and moods . . . the Digimon franchise, Digital World, and Digidestined/Tamers. **Hallucination: unrelated franchise memory.**	**Ours.** **Stored memory:** Czech grammar: aspect via prefix/suffix; indicative/imperative/conditional moods; phonemic orthography with hacek marks.

Table 5: Representative memory transition case studies across multiple baseline agents. Each row shows the input chunk, the memory stored by an existing method, and the memory stored by TRUSTMEM. Blue highlights mark relevant evidence in the input; red highlights mark corrupted or unsupported memory spans, and bold red annotations identify the failure type.

A Experiment Settings

Datasets. We evaluate TRUSTMEM on MemoryAgentBench (Hu et al., 2025), HaluMem (Chen et al., 2025), and the Mem-α validation set. Following Mem-α (Wang et al., 2025b), we use its training pool, which aggregates eight sources: SQuAD (Rajpurkar et al., 2016), HotpotQA (Yang et al., 2018), PerLTQA (Du et al., 2024), LongMemEval-Train (Wu et al., 2024), NLU (Larson et al., 2019), TREC-C (Li and Roth, 2006), PubMed-RCT (Dernoncourt and Lee, 2017), and BookSum (Kryściński et al., 2022). We train TRUSTMEM on a balanced subset of 562 instances sampled from the 4,139-instance Mem-α training pool, and evaluate validation performance and ablations on 463 held-out Mem-α validation instances. We use all datasets, models, and systems only for research purposes and follow their publicly released access conditions and licenses where available.

MemoryAgentBench. We evaluate TRUSTMEM on representative tasks from MemoryAgentBench (Hu et al., 2025) across three memory capabilities. For Accurate Retrieval (AR), we use Single-Doc, Multi-Doc, and LME(S), which evaluate whether the agent can retrieve useful information from historical contexts for downstream queries. For Test-Time Learning (TTL), we use five multi-class classification datasets: TREC-C (Li and Roth, 2006), NLU (Larson et al., 2019), TREC-F (Li and Roth, 2006), Clinic (Larson et al., 2019), and Banking77 (Casanueva et al., 2020), which test whether the agent can acquire new classification rules from examples during interaction. For Long-Range Understanding (LRU), we use InfBench-Sum (Zhang et al., 2024), which evaluates the agent's ability to integrate information across long contexts for summarization. Each task is evaluated with its original task-specific scoring protocol, and the final results are reported as task-level performance scores.

HaluMem Bench. HaluMem (Chen et al., 2025) is an operation-level benchmark focusing on memory safety, including hallucinations, omissions, and incorrect updates, during memory extraction, updating, and question answering. We use HaluMem-Medium, which contains 30,073 dialogue turns from 20 users ($\sim$160K tokens/user), encompassing 14,948 memory points and 3,467 QA pairs. While MemoryAgentBench evaluates downstream memory utility and out-of-distribution generalization, HaluMem evaluates memory update safety and operation-level reliability through stage-wise extraction, updating, and QA metrics.

Baselines. We compare TRUSTMEM with a broad set of baselines covering three memory-processing paradigms, as summarized in Tables 6. (1) *Standard paradigms*: **Long-Context** directly places the interaction history into the context window, while **RAG-Top2** retrieves the top-2 relevant chunks before answer generation. These baselines test whether long-context encoding or simple retrieval is sufficient for long-term memory tasks. (2) *RL-based and agentic memory methods*: We compare with recent methods that train or design agents to construct, update, and use long-term memory, including **Mem-α** (Wang et al., 2025b), **MemAgent** (Yu et al., 2025), **MEM1** (Zhou et al., 2025), **Mem-T** (Yue et al., 2026), and **AtomMem** (Huo et al., 2026). These methods typically process long inputs incrementally and maintain compact or structured memory states for downstream tasks. (3) *System-level memory frameworks*: We further compare with practical memory management systems, including **Mem0** (Chhikara et al., 2025), **Memobase** (Ye et al., 2025), **Supermemory** (Shah et al., 2025), and **Zep** (Rasmussen et al., 2025), which provide memory extraction, updating, retrieval, and memory-based question answering capabilities. Since different benchmarks emphasize different aspects of memory behavior and not all baselines are applicable to every setting, we report the available and comparable results for each benchmark in the corresponding tables.

Implementation Details. TRUSTMEM is initialized from Qwen3-4B (Yang et al., 2025) and trained only on the Mem-α training pool with Transition-Ranked GRPO for approximately two days on 32 NVIDIA H100 GPUs. No examples from MemoryAgentBench or HaluMem are used for training; all results are reported on held-out validation instances or external benchmarks. We report results from the step-85 checkpoint. Following the reward definition in Section 3, the training reward is defined as $R = R_{\text{task}} + R_{\text{act}} + 0.1R_{\text{ver}} + 0.05R_{\text{eff}} + 0.1R_{\text{type}}$, where R_{task} measures downstream task performance, R_{act} evaluates action executability, R_{ver} measures transition-level memory trustworthiness, R_{eff} encourages compact memory construction, and R_{type} evaluates content specificity. We use 8 GRPO rollouts per instance, a learning rate of 1×10^{-6}, KL coefficient 0.001, maximum prompt length 4096, maximum response length 2048. For the preference-ranking objective,

we use top-1/bottom-1 pair construction with score margin 0.15, at most 256 preference pairs per batch, pair micro-batch size of 1 per GPU, preference loss weight $\lambda_{\text{rank}} = 0.05$, and $\beta_{\text{rank}} = 0.1$. Thinking mode is disabled during both training and evaluation.

Evaluation Metrics. For the MemoryAgentBench and validation set , we follow the original evaluation protocols of the corresponding benchmarks. Since different data sources use different scoring functions, we evaluate each instance with its dataset-specific metric and report the resulting task-level performance score for each dataset. The Avg. column denotes the macro-average over the reported task scores. For HaluMem, we report operation-level metrics over three stages: memory extraction, memory updating, and memory-based question answering. For memory extraction, we report recall (R), weighted recall (Weighted R), accuracy (Acc.), and F1. For memory updating and question answering, we report correctness (C), hallucination rate (H), and omission rate (O). Higher values indicate better performance for R, Weighted R, Acc., F1, and C, while lower values are better for H and O.

The exact subset of baselines evaluated on each benchmark depends on the applicability of the corresponding method and the availability of comparable results. The reported baselines for each benchmark are shown in the corresponding result tables.

B Experiments on Memory Utility

Table 1 reports results on the validation set. TRUSTMEM achieves the best average score of 66.3, outperforming Mem-α by 4.4 points. The improvement is especially clear on Test-Time Learning, where TRUSTMEM obtains 72.4 on TREC-C, 73.8 on NLU, and 64.0 on PubMed. TRUSTMEM also improves BookSum from 18.7 to 22.6, showing stronger long-range memory utility. These results suggest that transition-level verification helps the agent preserve useful task-specific and long-range information during memory construction.

Table 2 evaluates out-of-distribution generalization on MemoryAgentBench. TRUSTMEM achieves the best average score of 65.7, improving over Mem-α by 6.5 points and outperforming Long-Context and RAG-Top2 by 19.6 and 15.5 points, respectively. TRUSTMEM performs strongly on both retrieval and test-time learning tasks, achieving 83.0 on Single-Doc, 85.0 on TREC-C, and 84.0 on Banking77. These results show that TRUSTMEM learns a more robust memory update policy than simply retaining long context or retrieving top-ranked chunks. Overall, reliable local memory transitions improve both validation performance and out-of-distribution memory utility.

C Experiments on HaluMem

Table 3 evaluates operation-level memory reliability on HaluMem-Medium, covering memory extraction, memory updating, and memory-based question answering. This benchmark directly matches the goal of TRUSTMEM: improving the safety and reliability of each local memory transition.

For memory extraction, TRUSTMEM achieves the best recall, weighted recall, and F1, with scores of 56.73, 67.45, and 69.45, respectively. Compared with the strongest prior F1 score from Mem0, TRUSTMEM improves extraction F1 by 12.14 points. Although AtomMem obtains the highest extraction accuracy, its recall and F1 are extremely low, indicating that it misses most useful memories. In contrast, TRUSTMEM achieves a better balance between coverage and correctness, showing that the verifier helps reduce both omission and unsupported memory writing.

For memory updating, TRUSTMEM achieves the lowest omission rate of 48.64 and the second-best correctness score of 46.84. This supports the role of the Memory Transition Verifier, which checks whether each update preserves valid prior memory, incorporates new evidence, and avoids unsupported changes. As a result, TRUSTMEM learns safer memory-editing behavior and reduces missed updates.

For memory-based question answering, TRUSTMEM obtains the lowest omission rate of 12.46 and the second-lowest hallucination rate of 6.07 while maintaining competitive correctness. Compared with methods that either hallucinate more, such as Mem-T, or omit too much information, such as AtomMem, TRUSTMEM provides a stronger trade-off across correctness, hallucination, and omission. Overall, these results show that transition-level verification and preference-guided optimization improve not only final task performance but also the intermediate memory operations required for trustworthy long-term memory.

Method	Category	Model/System Used	Backbone/Base Model
Long-Context	Standard paradigm	Qwen3-32B with 32K context window	Qwen3-32B
RAG-Top2	Standard paradigm	BM25 top-2 retrieval + Qwen3-32B	Qwen3-32B answer model
MemAgent (Yu et al., 2025)	RL-based / agentic memory method	RL-MemAgent-7B	Qwen2.5-7B
MEM1 (Zhou et al., 2025)	RL-based / agentic memory method	MEM1-7B	Qwen2.5-7B
Mem-α (Wang et al., 2025b)	RL-based memory method	Mem-α	Qwen3-4B
Mem-T (Yue et al., 2026)	RL-based / agentic memory method	Mem-T-4B	Qwen3-4B
AtomMem (Huo et al., 2026)	RL-based / agentic memory method	AtomMem-8B	Qwen3-8B
Mem0 (Chhikara et al., 2025)	System-level memory framework	Mem0	Not fixed
Memobase (Ye et al., 2025)	System-level memory framework	Memobase	Not fixed
Supermemory (Shah et al., 2025)	System-level memory framework	Supermemory	Not fixed
Zep (Rasmussen et al., 2025)	System-level memory framework	Zep	Not fixed

Table 6: Summary of baseline models and systems used in our experiments. For model-based baselines, we report the evaluated checkpoint or model and its backbone when applicable. For system-level memory frameworks, the backbone is marked as "Not fixed" because these methods function as memory infrastructures and can be paired with different LLMs during extraction, updating, retrieval, or answer generation.

Prompt Template for Transition Verification

Role: You are an expert judge for chunk-level memory consolidation.

Goal: Given a compact view of one memory transition from M_i to M_{i+1} after processing a chunk c_i, evaluate whether the memory update is trustworthy according to:

1. **Coverage:** whether M_{i+1} preserves the write-worthy key information from c_i;
2. **Preservation:** whether important information already present in M_i is not deleted or overwritten without justification;
3. **Faithfulness:** whether newly added or modified memory content is supported by the chunk evidence or the previous memory state.

Inputs:

- **Chunk Evidence:** `{chunk_text}`
- **Pre-transition Memory State:** `{pre_memory}`
- **Post-transition Memory State:** `{post_memory}`
- **Touched Memory Actions:** `{actions}`

Required Output: Return only a JSON object:

```
{
  "score": 0.0,
  "coverage_ok": true,
  "preservation_ok": true,
  "faithfulness_ok": true,
  "issues": ["brief issue labels"],
  "explanation": "brief explanation"
}
```

Rules:

- Judge only from the provided chunk evidence, pre-transition memory, post-transition memory, and touched memory actions.
- A skipped action can still be valid if the relevant information was already correctly stored and preserved.
- Legitimate merging, rewriting, or compression is allowed as long as important facts are preserved.
- Penalize critical omissions, unsupported additions, corrupted facts, and unjustified overwrites or deletions.
- Use a continuous score in $[0, 1]$, where 1 means excellent consolidation and 0 means clearly invalid consolidation.
- Do not output extra text outside the required JSON object.

Figure 5: Simplified prompt template for transition verification.

D Memory Transition Error Judge Prompt

We use GPT-4o-mini (Hurst et al., 2024) with temperature set to 0 as the LLM judge for transition-level memory reliability evaluation. The judge receives the prior memory, the current input chunk, the memory operations produced after reading the chunk, and the updated memory, and returns a structured JSON judgment over omission, corruption, and hallucination.

Evaluation Protocol. For the reliability analysis in Section 4.4, we construct one record for each chunk-level memory transition. Each record contains the prior memory, the current input chunk, the generated memory-editing operations, and the updated memory. A transition may contain multiple error types, so we compute each error rate independently as the percentage of transitions marked with that error type. We count only errors marked as `major`; transitions with `minor` or `none` severity are not counted as failures.

Prompt Template for Memory Transition Error Judge

Role: You are a strict but fair evaluator of memory-transition quality.
Goal: Given one chunk-level memory transition, judge whether the transition introduces a substantive memory error. The judge evaluates three semantic failure modes:

1. **Omission:** the updated memory misses key information from the input chunk.
2. **Corruption:** the memory operation changes, contradicts, misattributes, or mixes a fact supported by the input chunk or prior memory.
3. **Hallucination:** the memory operation introduces a substantive new fact unsupported by the input chunk or prior memory.

Inputs:

- **Dataset Group:** `{dataset_group}`
- **Sample ID:** `{sample_id}`
- **Chunk Index:** `{chunk_idx}`
- **Prior Memory:** `{prior_memory}`
- **Input Chunk:** `{input_chunk}`
- **Memory Operations:** `{memory_operations}`

Definitions:

- **Key information** includes facts, labels, constraints, preferences, entities, relations, dates, numbers, or outcomes needed for later question answering, classification, personalization, or faithful recall.
- A concise summary is allowed; skipping nonessential details alone is not counted as omission.
- Facts already supported by prior memory are not counted as hallucinations when they are simply preserved by an update.

Required Output: Return only a JSON object:

```
{
  "omission": true,
  "corruption": false,
  "hallucination": false,
  "primary_error": "omission",
  "severity": "major",
  "confidence": 85,
  "rationale": "brief explanation",
  "evidence": ["supporting evidence strings"]
}
```

Rules:

- Judge only from the provided prior memory, input chunk, and memory operations.
- Do not penalize harmless meta text, timestamps, assistant acknowledgements, or memory bookkeeping unless it changes a substantive fact.

- For classification/example chunks, labels and representative examples are important.
- For long document chunks, central entities, events, relations, and numeric/date facts are important; minor wording differences are not errors.
- For omission, ask whether a future assistant would likely answer incorrectly or fail the benchmark because the key information was not stored.
- If there are multiple major error types, set `primary_error` to `"mixed"`.
- Use `"major"` only when the error would plausibly harm later recall or QA; use `"minor"` for small issues that probably would not matter.
- Use `"none"` when no substantive error is present.
- Do not output extra text outside the required JSON object.